ВСТУП

Текстові файли є одним з видів файлів, призначених для зберігання текстової та числової інформації. Використання файлів цього виду в програмах і отримання навичок читання даних з таких файлів і записування інформації у них відносяться до азів програмування.

Метою даної лабораторної роботи є освоєння методики використання текстових файлів у програмах, написаних мовою C++.

1. ТЕОРЕТИЧНІ ОСНОВИ

Зовнішнє ім’я файлу зазначається з урахуванням методики його подання для операційної системи. Воно може бути подане в повному вигляді (диск, шлях, ім’я: `"C:\\Users\\a.txt"`) або в скороченому – з урахуванням принципів умовчання, що закладені в роботу операційної системи, а саме:

- якщо не зазначене ім’я диска, то розглядається активний диск;
- якщо не зазначений шлях до файлу, то розглядається поточна папка (поточний каталог);
- якщо шлях починається не із символу `\`, то відлік шляху починається з поточної папки (слід пам’ятати про те, що в C++ символ `\` є управляючим і в разі необхідності його подання він подвоюється: `'\\'`).

1.1. Потокове введення/виведення

Потік розглядається як послідовність байт (символів) і на залежить від конкретного пристрою. Потокові введення і виведення можливі для друкувального пристрою, дисплея і дискових файлів, забезпечуючи при цьому буферизацію процесів уведення і виведення.

При роботі з файлами потік можна розглядати як файл разом із засобами буферизації.

Використовуючи потоки можна:

- створювати файли і знищувати файли;
- відкривати і закривати файли;
- приєднувати файл до потоку і від’єднувати потік від файлу;

- здійснювати обмін даними між пам’яттю і файлом за допомогою потоку;
- аналізувати помилки потокового введення/виведення і умову досягнення кінця потоку (файлу);
- керувати буферизацією;
- керувати покажчиком поточної позиції у потоці.

Якщо потік відкритий, то з ним зв’язується так званий покажчик (індикатор) поточної позиції. Читання/записування даних здійснюється починаючи з того байту файлу, на який вказує цей покажчик. При цьому покажчик поточної позиції автоматично «переміщається» по потоку на ту кількість байт, що відповідає зчитаному/записаному обсягу інформації. Результатом є те, що наступний сеанс читання/записування здійснюватиметься з орієнтацією на нове положення покажчика поточної позиції.

1.2. Робота з текстовими файлами з використанням потоків

Одним із видів файлів, якими може оперувати програма, написана мовою C++, є текстові файли, що складаються з символьних рядків змінної довжини, завершуваних спеціальною комбінацією символів, називаною «кінцем рядка». Комбінація «кінець рядка» складається з двох символів – «повернення каретки» (`'\xD'`, десятковий код 13) і «переведення рядка» (`'\xA'`, десятковий код 10). Крім фізичного кінця файлу, як ознака кінця файлу в текстових файлах сприймається символ «кінець файлу» (`'\x1A'`, десятковий код 26). Такий файл можна підготувати будь-яким текстовим редактором, у тому числі редактором, що вбудований в інтегроване середовище Visual Studio.

Якщо потік відкритий у текстовому режимі, то прочитана з нього послідовність символів «повернення каретки» і «переведення рядка» перетворюється на символ нового рядка `'\n'` (значення 10). При записуванні в потік здійснюється зворотне перетворення.

У C++ потоки для роботи з файлами створюються як об’єкти наступних класів:

`ofstream` – для виведення (записування) даних у файл;

`ifstream` – для введення (читання) даних з файлу;

`fstream` – для читання і для запису даних (двонаправлений обмін).

Щоб використовувати ці класи, у текст програми необхідно включити додатковий заголовний файл `fstream`. Після цього в програмі можна визначати конкретні файлові потоки відповідних типів (об'єкти класів `ofstream`, `ifstream`, `fstream`), наприклад, так:

```
ifstream inFile;          // Визначається вхідний
                                  // файловий потік
ofstream outFile;        // Визначається вихідний
                                  // файловий потік
fstream ioFile;    // Визначається файловий потік
                        // для введення і виведення
```

Створення файлового потоку (об'єкту відповідного класу) зв'язує ім'я потоку з виділеним для нього буфером і ініціалізує змінні стану потоку.

Створивши файловий потік, можна «приєднати» його до конкретного файлу за допомогою компонентної функції `open()`. З її допомогою можна не тільки відкрити файл, але і зв'язати його з вже визначеним потоком. Формат цієї функції такий:

```
void open( const char * ім'я_файлу,
           int режим = значення_за_умовчанням,
           int захист = значення_за_умовчанням );
```

Перший параметр – ім'я_файлу – це ім'я вже існуючого або створюваного наново файлу. Це рядок, що визначає повне або скорочене ім'я файлу у форматі, регламентованому операційною системою. Другий параметр – режим – це диз'юнкція прапорів, які визначають режим роботи з відкриваним файлом (наприклад, тільки запис або тільки читання). Нижче наведено визначені в класі `ios` (що у свою чергу визначений у просторі імен `std`) прапори режиму роботи для текстових файлів:

`ios::in = 0x01` – відкрити існуючий файл тільки для читання;

`ios::out = 0x02` – відкрити новий файл тільки для записування (якщо файл з ім'ям, указаним у третьому параметрі, вже існує, то він буде знищений і створений наново);

`ios::ate = 0x04` – при відкритті потоку покажчик поточної позиції встановлюється на кінець файлу; читати/писати можна в будь-якій позиції залежно від того, де розміщується покажчик поточної позиції, якщо його примусово перемістити;

`ios::app = 0x08` – завжди дописувати дані в кінець існуючого файлу (якщо файл з ім'ям, указаним у третьому параметрі, вже існує, то він буде створений);

`ios::trunc = 0x10` – замість існуючого створювати новий файл (режим діє за умовчанням, якщо не вказано `ios::in`, `ios::ate` або `ios::app`);

`ios::nocreate = 0x20` – не відкривати новий файл; якщо файл відсутній генерується помилка;

`ios::noreplace = 0x40` – не відкривати існуючий файл; для існуючого вихідного файлу при відсутності режимів `ios::ate` або `ios::app` генерується помилка.

Другий параметр (тільки разом з третім) може бути опущений. У цьому випадку береться його значення за умовчанням, що залежить від типу потоку, для якого викликається функція `open()`. Для `ifstream`-потоку таким значенням є `ios::in`, а для `ofstream`-потоку – `ios::out`. Для потоків класу `fstream` другий аргумент функції `open()` повинен бути заданий явно, оскільки неясно, у якому напрямку передбачається виконувати обмін з потоком (`ios::in` – тільки для читання, `ios::out` – тільки для запису, `ios::in | ios::out` – двонаправлений обмін для існуючого файлу, `ios::app | ios::in` – для дозаписування з можливістю читання).

Для потоків класів `ofstream` і `fstream`, якщо в режимі відкриття вказано прапор `ios::app`, незалежно від наявності прапорів `ios::out` та `ios::in` виведення здійснюється тільки в кінець файлу, не зважаючи на положення поточного покажчика потоку.

Третій параметр функції `open()` визначає захист файлу і звичайно встановлюється за умовчанням.

Виклик функції `open()`, здійснюється за допомогою уточненого імені:

ім'я_об'єкту_класу . виклик_функції_що_належить_класу

Таким чином, відкриття і приєднання файлу до конкретного файлового потоку забезпечується таким викликом функції `open()`:

ім'я_потоку.`open`(ім'я_файлу, режим, захист);

Тут ім'я_потоку – це ім'я одного з об'єктів, що належать класам `ofstream`, `ifstream` або `fstream`.

Приклади викликів функції `open()` для визначених вище потоків:

```
outFile.open("C:\\USERS\\result.txt");
inFile.open("data.txt");
```

```
ioFile.open("change.txt",ios::out);
```

При відкритті файлів з потоками класу `ofstream` другий параметр за умовчанням встановлюється рівним `ios::out`, тобто файл відкривається тільки для виведення. Таким чином, файл `C:\USERS\result.txt` після вдалого виконання функції `open()` буде при необхідності (якщо він не існував раніше) створений, а потім відкритий для виведення (запису) даних у текстовому режимі обміну і приєднаний до потоку `outFile`. Після цього до потоку `outFile` може застосовуватися, наприклад, операція включення <<. Так, оператор

```
outFile << -56 << '\t' << 3.1415;
```

записує два числа (ціле і дійсне) у файл `C:\USERS\result.txt` з розділенням їх символом табуляції.

Потік `inFile` класу `ifstream` у наведеному прикладі приєднується функцією `open()` до файлу з ім'ям `data.txt`. Цей файл відкривається для читання з нього даних у текстовому режимі. Якщо файл з ім'ям `data.txt` не існує, то спроба викликати функцію `inFile.open()` призведе до помилки.

До потоку `inFile` після його відкриття може застосовуватися операція витягання >>. Так, за умови, що в програмі визначені змінні `k` (наприклад, типу **`int`**) і d (типу **`double`**), оператор

```
inFile >> k >> d;
```

уводить з файлу `data.txt` два числа, записуючи їх у змінні `k` і `d` з автоматичним перетворенням до типів **`int`** і **`double`** відповідно.

Для перевірки вдалості завершення функції `open()` до потоку застосовують операцію `!`. Вираз `!ім'я_потоку` має ненульове значення за наявності помилки у відкритті потоку і дорівнює нулю, якщо помилки не було. Таким чином, результат виконання функції `open()` можна, наприклад, перевірити так:

```
if (!inFile)
{
    // Дії при помилці
}
```

Файл `change.txt` у прикладі, що наводився вище за текстом, відкривається для записування і зв'язується з потоком `ioFile`, який буде вихідним потоком до тих пір, поки за допомогою повторного відкриття

файлу явно не зміниться напрямок обміну з файлом або потоком. Щоб змінити режими доступу до файлу, його потрібно попередньо закрити за допомогою функції `close()`.

Закриття файлу, приєднаного до будь-якого потоку, здійснюється функцією без параметрів `close()`; наприклад:

```
ioFile.close();
```

Ця функція від'єднує файл від потоку і закриває його, унеможливлюючи доступ до нього без повторного відкриття. Якщо потік був відкритий для виведення, перед закриттям файлу здійснюється переписування в нього вмісту буферу.

При введенні даних з текстового файлу можуть використовуватися такі функції (перелік далеко не повний):

`void` `clear(int = 0);` – встановлює стан помилки потоку в нуль;

`bool` `eof();` – повертає значення **`true`** (ненульове значення), якщо має місце умова кінця файлу (раніше було прочитано ознаку кінця файлу). Будь-який символ, що залишався в потоці, у тому числі символ нового рядка (але не символ кінця файлу) генерує значення **`false`**;

`int` `get();` – передає із вхідного потоку наступний символ або значення `EOF`;

`int` `get(char& змінна);` – передає із вхідного потоку наступний символ і поміщає його в байт, на який указує параметр;

`int` `get(char *рядок, int макс_кількість, char обмежувач = '\n');` – передає із вхідного потоку не більше макс_кількість символів і поміщає ці символи в область пам'яті, на яку вказує перший параметр, з дописуванням до прочитаного рядка кінцевого символу `'\0'`; третій параметр визначає обмежувач, при досягненні якого припиняється читання (обмежувач не вилучається з файлу; третій параметр може бути опущений, що відповідає використанню як обмежувача символу `'\n'`);

`istream& getline(char *рядок, int макс_кількість, char обмежувач = '\n');` – аналогічна функції `get()` з тією ж кількістю параметрів, але символ-обмежувач вилучається із файлу без занесення в рядок;

`istream& ignore(int кількість, int роздільник);` – пропускає символи в кількості, заданій першим параметром, зупиняючись, якщо зустрінеться символ-роздільник;

int peek(); – читає з потоку (але не вилучає) наступний символ, повертаючи зчитаний символ або EOF, якщо має місце умова кінця файлу;

istream& read(**char** *рядок, **int** кількість); – читає задану другим параметром кількість символів (точніше не більше цього параметру), поміщаючи їх у масив, заданий першим параметром;

istream& seekg(streampos позиція); – установлює покажчик поточної позиції на абсолютну позицію, задану параметром;

istream& seekg(streamoff зсув, seek_dir відлік); – переміщає в потоці покажчик поточної позиції читання на число байт, задане першим параметром відносно другого параметру (0 == ios::beg – початок потоку, 1 == ios::cur – поточна позиція, 2 == ios::end – кінець потоку);

streampos tellg(); – повертає поточну позицію читання;

Деякі функції, що використовуються при виведенні:

ostream& put(**char** символ) – поміщає один символ у потік;

ostream& seekp(streampos позиція); – аналог seekg() з одним параметром;

ostream& seekp(streamoff зсув, seek_dir відлік); – аналог функції seekg() з двома параметрами;

ostream& write(**char** *рядок, **int** кількість); – поміщає в потік кількість символів з параметра рядок;

streampos tellp(); – повертає поточну позицію записування;

Відзначимо, що при оголошенні деяких з наведених вище функцій використані спеціальні типи, а саме, streampos (аналогом є тип ios::pos_type) і streamoff. Указані типи замінюють раніше використовуваний у відповідних функціях тип **long**, причому оговорюється, що використання замість них типу **long** є не досить коректним.

При введенні даних із файлу рекомендується перевіряти умову кінця файлу, оскільки читання ознаки його кінця призводить до помилки часу виконання. Для описаного вище потоку ioFile, зв'язаного із файлом change.txt, це можна зробити, наприклад, так:

```
while (ioFile.peek() != EOF)
{   // Дії по введенню і обробці елемента даних
}
```

Якщо файл відкритий і для введення, і для виведення (двонаправлений обмін), то в разі потреби переходу від уведення до виведення (або

навпаки) необхідно виконати примусове зміщення покажчика поточної позиції потоку (як варіант – на 0 байт відносно поточної позиції). Слід також відзначити, що при читанні ознаки кінця файлу подальша обробка файлу стає неможливою, і для забезпечення подальшого опрацьовування файлу необхідно встановлювати в нуль стан помилки потоку, для чого використовується функція `clear`. Особливо це актуально при двонаправленому обміні, коли після читання ознаки кінця файлу може виникнути необхідність, наприклад, у коригуваннівімісту файлу.

При роботі з файлами (будь-якими, а не тільки текстовими) корисними можуть бути такі дві функції, оголошені в заголовному файлі `io.h`:

`int` `remove(`**`const char`** `*шлях);` – знищення файлу, ім'я якого задане як параметр (файл повинен бути закритим);

`int` `rename(`**`const char`** `*старе_ім'я,` **`const char`** `* нове_ім'я);` – перейменування файлу або каталогу (файл повинен бути закритим). Якщо в новому імені файлу вказати інший шлях відносно старого, файл буде перенесений. Каталог перенести неможливо.

2. ПРИКЛАДИ ПРОГРАМ

Приклад 1. Дано текстовий файл, що містить декілька рядків довжиною не більше 255 символів. У кожному з рядків без помилок записано єдиний вираз, який має вигляд a#b, де a, b – невід'ємні цілочислові величини, # – одна з операцій +, –, /, *. У файлі за останнім виразом переведення рядка не виконувалося. Вивести кожен із виразів і їх значення на екран і в новий текстовий файл.

Розв'язок.

```
#include <cstdlib>
#include <conio.h>
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    int a, b;
    char s[256], c;
    int i;
    cout << "Input-file name? ";
    cin >> s;                              // Уводимо ім'я файлу
    cin.get();                  // Зчитуємо символ нового рядка
    ifstream fInp;
```

```
fInp.open(s);  // Відкриття потоку (тільки для читання)
if (!fInp)          // Якщо має місце помилка відкриття
{
    cout << "Error of opening of a input-file "
         << s;
    cout << "\nPress any key to exit.\n";
    getch();
    return 1;
}
cout << "Output-file name? ";
cin >> s;                          // Уводимо ім'я файлу
cin.get();              // Зчитуємо символ нового рядка
ofstream fOut;
fOut.open(s);   // Відкриття потоку (тільки для запису)
if (!fOut)          // Якщо має місце помилка відкриття
{
    cout << "Error of opening of a output-file "
         << s;
    cout << "\nPress any key to exit.\n";
    getch();
    return 2;
}
while (fInp.peek() != EOF)       // Поки не кінець файлу
{
    fInp.getline(s, 256);       // Читаємо рядок з файлу
    i = 0;
    a = 0;
    while (s[i] >= '0' && s[i] <= '9')    // Формування
    {                                     // першого операнда
        a = a * 10 + s[i] - '0';
        i++;
    }
    c = s[i++];                           // Знак операції
    b = 0;
    while (s[i] >= '0' && s[i] <= '9')    // Формування
    {                                     // другого операнда
        b = b * 10 + s[i] - '0';
        i++;
    }
    switch (c)             // Обчислення значення виразу
    {
        case '+': a += b;
```

```
                break;
            case '-': a -= b;
                break;
            case '/': a /= b;
                break;
            case '*': a *= b;
        }
        cout << s << " = " << a << endl;  // Вивід на екран
        fOut << s << " = " << a << endl;    // Вивід у файл
    }
    fOut.close();       // Закриття вихідного потоку (файлу)
    fInp.close();        // Закриття вхідного потоку (файлу)
    cout << "Press any key to exit.\n";
    getch();
    return 0;
}
```

Приклад 2. У текстовому файлі data.txt через довільну кількість будь-яких пробільних символів записано цілі числа, причому за останнім числом додаткові символи відсутні. Переписати у файл res.txt із файлу data.txt усі числа за винятком максимальних (у припущенні, що їх може бути декілька). У кожному рядку файлу res.txt (за винятком останнього) повинно міститися по 10 чисел із розділенням їх символом табуляції.

Розв'язок.

```
#include <conio.h>
#include <iostream>
#include <fstream>

using namespace std;
int main()
{
    int v, Max;
    bool b = true;
    ifstream fInp;
    fInp.open("data.txt");// Відкриття потоку (для читання)
    if (!fInp)          // Якщо має місце помилка відкриття
    {
        cout << "Error of opening of a input-file";
        cout << "\nPress any key to exit.\n";
        getch();
        return 1;
    }
```

```
ofstream fOut;
fOut.open("res.txt");  // Відкриття потоку (для запису)
if (!fOut)          // Якщо має місце помилка відкриття
{
    cout << "Error of opening of a output-file";
    cout << "\nPress any key to exit.\n";
    getch();
    return 2;
}
while(!fInp.eof())              // Поки не кінець потоку
{                                   // читаємо по одному
    fInp >> v;                                  // числу
    if (b)            // і шукаємо максимальне значення
    {
        b = false;
        Max = v;
    }
    else
        if (v > Max)
            Max = v;
}
fInp.close();                               // Закриваємо
                    // і знову відкриваємо вхідний потік
fInp.open("data.txt");
 // Замість закриття і повторного відкриття файлу можна
    // встановити стан потоку в нуль функцією clear() і
 // зсунути покажчик поточної позиції на початок потоку
//    fInp.clear();
//    fInp.seekg(0);
int i = 0;
while(!fInp.eof())
{
    fInp >> v;                      // Читаємо одне число
    if (v < Max)                   // Якщо це не максимум
    {
        if (i == 10)              // Оформлюємо виведення
        {
            fOut << endl;
            i = 0;
        }
        else
            if (i != 0)
```

```
                    fOut << '\t';
            fOut << v;                      // Виводимо число
            i++;
        }
    }
    fOut.close();               // Закриваємо вихідний потік
    fInp.close();                // Закриваємо вхідний потік
    cout << "Press any key to exit.\n";
    getch();
    return 0;
}
```

Приклад 3. Створити текстовий файл «Заміри температури пацієнтів поліклініки». Запис про кожного пацієнта заносити в окремий рядок файлу у вигляді: прізвище (послідовність не більш ніж з 30 символів без пробільних символів), вік (ціле число), температура (дійсне число) з розділенням цих елементів даних одним символом табуляції. Занести у файл інформацію про 10 пацієнтів.

Розв'язок.

```
#include <iostream>
#include <fstream>
#include <conio.h>
using namespace std;
int main()
{
    char FileName[100], Surname[31];
    int Age;
    double Temperature;
    cout << "Enter file name: ";
    cin >> FileName;                    // Уводимо ім'я файлу
    cin.get();                // Зчитуємо символ нового рядка
    ofstream fOut;   // Визначаємо вихідний потік і відкри-
    fOut.open(FileName);     // ваємо його тільки для запису
    if (!fOut)           // Якщо має місце помилка відкриття
    {
        cout << "Error of opening of a output-file "
             << FileName;
        cout << "\nPress any key to exit.\n";
        getch();
        return 1;
    }
    for (int i = 0; i < 10; i++)
```

```
    {
        cout << "Enter Surname: ";
        cin >> Surname;
        cout << "Enter Age: ";
        cin >> Age;
        cout << "Temperature: ";
        cin >> Temperature;
        fOut << Surname  << '\t' << Age <<  '\t'
             << Temperature << endl;       // Пишемо у файл
    }
    fOut.close();                          // Закриваємо потік
    puts("Press any key to exit.");
    getch();
    return 0;
}
```

Приклад 4. Дано текстовий файл «Заміри температури пацієнтів поліклініки». Запис про кожного пацієнта занесена в окремий рядок файлу у вигляді: прізвище (послідовність не більш ніж з 30 символів без пробільних символів), вік (ціле число), температура (дійсне число) із розділенням цих елементів даних у рядку файлу довільною кількістю будь-яких пробільних символів. Вивести на екран прізвища і температуру пацієнтів, чия температура, нижче 36.0 або вище 36.9 градусів.

Розв'язок.

```
#include <iostream>
#include <fstream>
#include <conio.h>
using namespace std;
int main()
{
    char FileName[100], Surname[31];
    int Age;
    double Temperature;
    cout << "Enter file name: ";
    cin >> FileName;                    // Уводимо ім'я файлу
    cin.get();                 // Зчитуємо символ нового рядка
    ifstream fInp;     // Визначаємо вхідний потік і відкри-
    fInp.open(FileName);   // ваємо його тільки для читання
    if (!fInp)           // Якщо має місце помилка відкриття
    {
        cout << "Error of opening of a input-file "
             << FileName;
```

```
        cout << "\nPress any key to exit.\n";
        getch();
        return 1;
    }
    while (!fInp.eof())              // Поки не кінець файлу
    {                                          // Читаємо з файлу
       fInp >> Surname  >> Age >> Temperature;
       if (Temperature < 36.0 || Temperature > 36.9)
           cout << Surname << '\t' << Temperature << endl;
                    // Якщо прочитаний рядок файлу містить
       if (fInp.peek() == '\n')      // ознаку кінця рядка,
           fInp.ignore(2);       // то пропускаємо 2 символи
    }
    fInp.close();                          // Закриваємо потік
    puts("Press any key to exit.");
    getch();
    return 0;
}
```

Приклад 5. Дано текстовий файл із рядками довжиною не більше 255 символів. Розвернути кожен його рядок на 180°.

Розв'язок.

```
#include <iostream>
#include <string.h>
#include <conio.h>
#include <fstream>
using namespace std;
int main()
{
   char FileName[100], str[256];
   cout << "File name? ";
   cin >> FileName;                    // Уводимо ім'я файлу
   fstream fInpOut;      // Визначаємо двонаправлений потік
         // і зв'язуємо його з існуючим файлом для введення
   fInpOut.open(FileName, ios::in | ios::out);  // і виводу
   if (!fInpOut)
   {
      cout << "Error of opening of a file " << FileName;
      cout << "\nPress any key to exit.\n";
      _getch();
      return 1;
   }
```

```
   streampos Pos1 = 0; // Поточна позиція файлу (початкова)
   while (fInpOut.peek()!=EOF)
   {
      fInpOut.getline(str, 256);           // Читаємо рядок
               // Зараз активний режим читання. Далі писати
            // не можна. Покажчик поточної позиції зсунувся
       // на strlen(str) байт, якщо було прочитано останній
       // рядок, за яким у файлі вісутня ознака кінця рядка
             // (буде прочитано ознаку кінця файлу), або на
              // strlen(str) + 2 байт для усіх рядків, крім
       // останнього (у файлі ці рядки завершуються ознакою
           // кінця рядка, яка позначається двома символами
                    // і також зчитується функцією getline)
       fInpOut.clear();     // Встановлюємо встановлює стан
           // помилки потоку в нуль, оскільки після читання
       // останнього рядка файлу буде прочитана ознака кінця
        // файлу і подальша робота з файлом буде неможливою
          // Запам'ятовуємо положення поточного покажчика
       streampos Pos2 = fInpOut.tellg();
            // Розвертаємо вміст рядка
      for (int i = 0, k = strlen(str); i < k / 2; i++)
      {
         char ch = str[i];
         str[i] = str[k - 1 - i];
         str[k - 1 - i] = ch;
      }
      fInpOut.seekp(Pos1, ios::beg);          // Повертаємо
              // покажчик поточної позиції у положення, яке
           // він займав перед читанням. Тепер можна писати
      fInpOut << str;// Пишемо у файл. Далі читати не можна
      Pos1 = Pos2;
      fInpOut.seekg(Pos2);        // Встановлюємо покажчик
               // у положення, яке він займав по закінченні
         // попереднього сеансу читання. Тепер можна читати
   }
   fInpOut.close();                       // Закриваємо потік
   puts("Press any key to exit.");
   _getch();
   return 0;
}
```

3. ВАРІАНТИ ЗАДАЧ

1. Дано текстовий файл. Переписати в інший файл усі рядки початкового файлу, що не містять латинських літер.
2. Дано текстовий файл. Переписати в інший файл частини рядків, починаючи з останнього слова[1], що не містить цифри.
3. Дано текстовий файл із рядками довжиною не більше 255 символів. Переписати його вміст в інший текстовий файл за рядками, починаючи з останнього рядка і закінчуючи першим.
4. Дано текстовий файл. Вивести найдовше слово тексту, що міститься в ньому. Якщо таких слів декілька, вивести:
   а) останнє з них;
   б) перше з них.
5. Дано натуральне число *n*. Записати в текстовий файл усі подання цього числа сумою двох натуральних чисел. Перестановка доданків нового способу не дає.
6. Дано текстовий файл із довжинами рядків не більше 255 символів. Переписати в інший файл через пробіл всі числа, які представлені в початковому файлі як окремі слова. Довжини рядків у результуючому файлі не повинні перевищувати 255. У разі невиконання цієї умови, продовжувати виведення в новому рядку.
7. Дано текстовий файл. Переписати в інший файл через один пробіл тільки неповторювані слова вихідного файлу.
8. Дано текстовий файл. Переписати в інший файл його рядки відповідно до зростання їх довжин.
9. Дано текстовий файл з текстом, написаним англійською мовою. Переписати в інший файл його вміст за таким правилом: якщо у файлі зустрілася крапка, то наступний текст повинен сприйматися як нове речення, що починається з великої літери; усі символи «крапка з комою» повинні бути замінені крапками з відповідним перетворенням наступного тексту.
10. Дано текстовий файл. Переписати в інший файл усі рядки, що не містять слова-«перевертні» (паліндроми). У результуючому файлі рядки повинні йти в порядку, зворотному порядку рядків початкового файлу.

[1] Тут і далі під словом будемо розуміти будь яку послідовність символів, що не містить пробільні символи і символи керування, тобто символів з кодами, меншими за 33.

11. Дано текстовий файл. Переписати в інший файл у порядку, зворотному початковому, всі рядки першого файлу, розгорнувши в них усі слова на 180°.
12. Вводячи дані з клавіатури, сформувати текстовий файл fish.txt, рядки якого містять відомості «Суднового добового донесення»: назва судна, бортовий номер судна, дата вилову риби, назва риби, вага вилову (у тоннах). Дано дійсне число *R* і текстовий рядок *S*. Вивести на екран рядки файлу fish.txt, у яких вага вилову перевищує *R* тон. Сформувати текстовий файл selection.txt, записавши в нього рядки файлу fish.txt, що задовольняють умові: назва риби збігається з *S*.
13. Дано два текстових файли. Чи мають їх відповідні рядки однакову довжину?
14. Дано текстовий файл, що містить деякий текст із декількох абзаців, кожен з яких починається з нового рядка, позначуваного символом (символами) табуляції або (та) одним чи декількома пробілами. Не вважаючи за абзаци порожні рядки і рядки, що містять лише пробіли та символи табуляції, визначити кількість абзаців у тексті.
15. Дано текстовий файл f, що містить деякий текст із кількох абзаців, кожен з яких починається з нового рядка, позначуваного більш ніж одним пробілом. Файл f не містить порожніх рядків і рядків, що складаються з одних пробілів. Переписати в новий файл уміст файлу f, вставити між сусідніми абзацами по одному порожньому рядку і видаливши у всіх рядках початкові пробіли.
16. Дано текстовий файл f. Переписати його вміст у файл *g*, вставивши в початок кожного рядка його номер, відлічуваний від 1. Номер відділяти від умісту рядка одним пробілом.
17. Дано текст, записаний у текстовому файлі f. Переписати у файл *g* уміст файлу f, виключивши повторення слів. У третій файл (файл *h*) записати по одному слову в рядку ті слова, що не були записані у файл *g*. У файлі *g* не повинно бути порожніх рядків. Варіанти:

а) Слова, що повторюються у файлі f, не повинні потрапляти у файл *g*.

б) Повторюване у файлі f слово має бути записане у файл *g* один раз у тому місці, де воно зустрілося вперше у файлі f.

18. Дано два текстових файли f_1 і f_2. Переписати вміст файлу f_1 у новий файл f_3, додавши до кожного рядка файлу f_1 відповідний рядок файлу

f_2. Якщо файл f_2 вичерпається раніше, ніж файл f_1, то рядки файлу f_1, що залишилися, переписати без їх змінювання. Додавання рядка виконувати через один пробіл:

а) у початок відповідного рядка файлу f_1;

б) у кінець відповідного рядка файлу f_1.

Навчальне видання

Методичні вказівки
до лабораторної роботи
«Текстові файли в програмах мовою С++»
з курсу «Програмування» для студентів напряму
6.040302 – Інформатика і курсу «Програмування
та алгоритмічні мови» для студентів напряму
6.040303 – Системний аналіз

Укладачі: БЕЗМЕНОВ Микола Іванович,
БЕЗМЕНОВА Ольга Миколаївна

Відповідальний за випуск О. С. Куценко
Роботу до видання рекомендував О. В. Горілий

За авторською редакцією

План 2014 р., поз. 105.

Підп. до друку 07.07.2014 р. Формат 60×84 1/16. Папір офсетний.
Riso-друк. Гарнітура Таймс. Ум. друк. арк. 1,2. Наклад 50 пр.
Зам. № 251. Ціна договірна.

Видавець і виготовлювач
Видавничий центр НТУ «ХПІ»,
вул. Фрунзе, 21, Харків, 61002.

Свідоцтво суб’єкта видавничої справи ДК № 3657 від 27.12.2009 р.